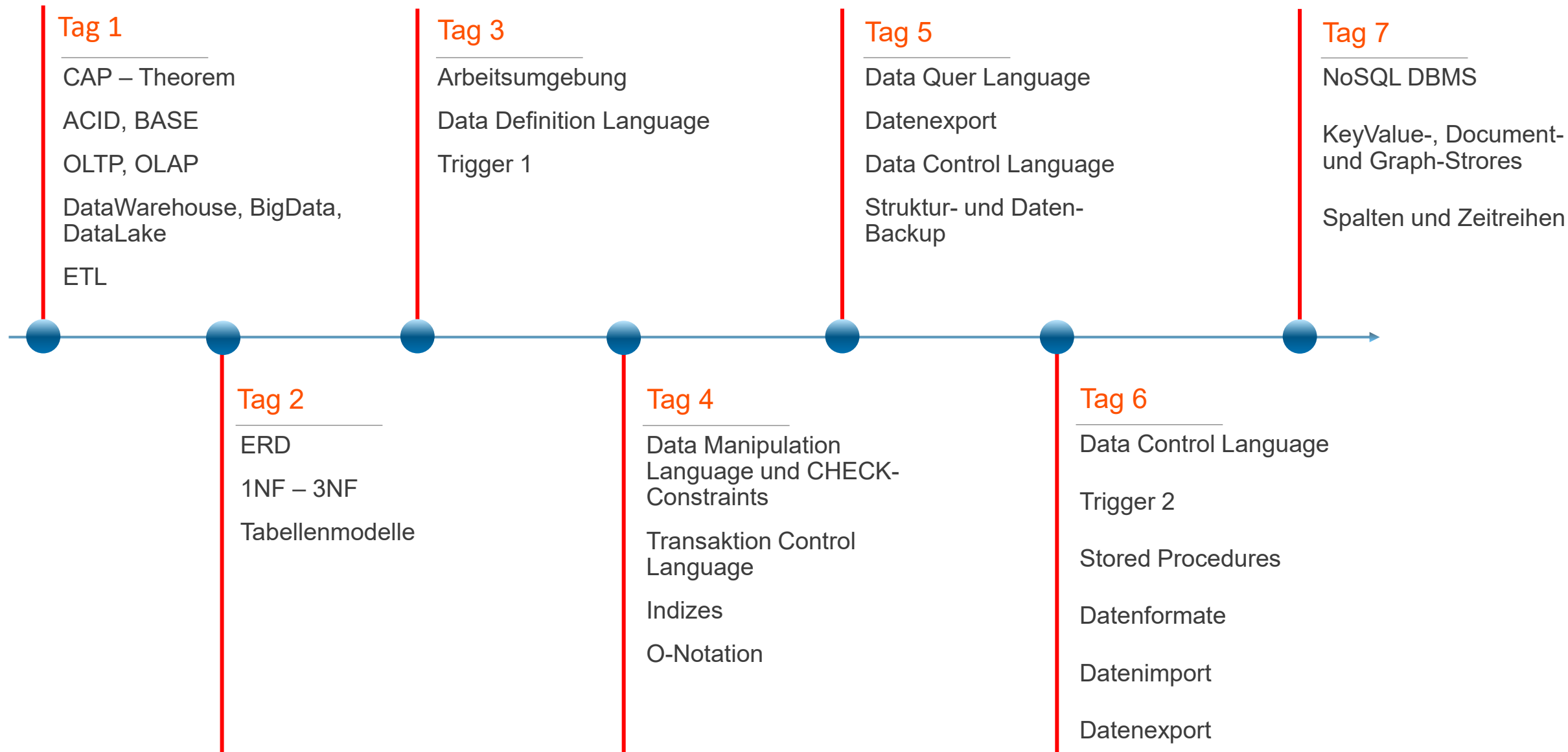


DB2



1

2

.....
.....

3

.....
.....

4

.....
.....

DML Data Manipulation Language

Fortsetzung DML



Data Manipulation Language

Was bisher geschah

Wir haben:

- die Datenbank MariaDB und den Datenbank-Client HeidiSQL lokal installiert.
- die nötigen Grundeinstellungen durchgeführt.
- die Datenbank ddl_db und die Tabelle mitarbeiter angelegt.

Jetzt ist es an der Zeit unsere Datenbank mit Leben zu füllen. Das bedeutet, dass wir endlich auch Daten einfügen werden.

Szenario:

- Wir haben von der Personalabteilung die erforderlichen Daten von fünf Mitarbeitern bekommen, welche wir jetzt in die Tabelle mitarbeiter einpflegen.
- Hierbei erfahren wir von der Personalabteilung, dass ein Mitarbeiter entweder Gehalt oder Stundenlohn auf Basis der zu leistenden Stunden erhält. Es muss vermieden werden, dass sowohl in dem Attribut Gehalt als auch in den Attributen stundenlohn und wochenstunden Werte stehen. Das würde zu einem Chaos bei der Lohnabrechnung führen.

Testdaten eingeben

Da wir versierte Datenbank-Benutzer sind wollen wir zuerst die Testdaten eingeben und uns dann den erforderlichen CONSTRAINTS widmen.

```
1  -- Mitarbeiter mit Gehalt (MA 1, 2 und 4)
2  INSERT INTO mitarbeiter (
3      mitarbeiter_nummer, vorname, nachname, abteilung,
4      vorgesetzter_id, eintritts_datum, gehalt, stundenlohn,
5      wochenstunden, geschlecht, interessen,
6      angelegt_am, angelegt_von, letzte_aenderung_von
7  ) VALUES
8  (
9      'MA-001', 'Max', 'Mustermann', 'IT-Leitung',
10     NULL, '2023-01-01', 6500.00, NULL, NULL, 'm', 'Gaming,Musik',
11     NOW(), 'Admin_System', '-'
12 ),
13 (
14     'MA-002', 'Erika', 'Schmidt', 'Personalwesen',
15     1, '2023-02-15', 4800.00, NULL, NULL, 'w', 'Lesen,Reisen',
16     NOW(), 'Admin_System', '-'
17 ),
18 (
19     'MA-004', 'Sarah', 'König', 'Marketing',
20     NULL, '2023-08-10', 4500.00, NULL, NULL, 'w', 'Kochen & Bakken',
21     NOW(), 'Admin_System', '-'
22 );
```

```
24 -- Mitarbeiter mit Stundenlohn (MA 3 und 5)
25 INSERT INTO mitarbeiter (
26     mitarbeiter_nummer, vorname, nachname, abteilung,
27     vorgesetzter_id, eintritts_datum, gehalt, stundenlohn,
28     wochenstunden, geschlecht, interessen,
29     angelegt_am, angelegt_von, letzte_aenderung_von
30 ) VALUES
31 (
32     'MA-003', 'Alex', 'Meyer', 'IT-Support',
33     1, '2023-05-01', NULL, 30.00, 40, 'd', 'Fotografieren,Gaming',
34     NOW(), 'Admin_System', '-'
35 ),
36 (
37     'MA-005', 'Thomas', 'Bauer', 'Marketing',
38     4, '2023-11-20', NULL, 24.50, 20, 'm', 'Sport & Fitness,Musik',
39     NOW(), 'Admin_System', '-'
```

INSERT()

Unser INSERT-Statement hat ganze Arbeit geleistet und die Daten in der gewünschten Form in die Tabelle eingetra-gen.

1 id	2 mitarbeiter_nummer	3 vorname	4 nachname	5 abteilung	6 vorgesetzter_id	7 eintritts_datum	8 austritts_datum	9 gehalt	10 stundenlohn	11 wochenstunden	12 geschlecht	13 interessen	
1	1	MA-001	Max	Mustermann	IT-Leitung	(NULL)	2023-01-01	(NULL)	6.500,0	(NULL)	(NULL)	m	Gaming,Musik
2	2	MA-002	Erika	Schmidt	Personalwesen	1	2023-02-15	(NULL)	4.800,0	(NULL)	(NULL)	w	Lesen,Reisen
3	3	MA-004	Sarah	König	Marketing	(NULL)	2023-08-10	(NULL)	4.500,0	(NULL)	(NULL)	w	Kochen & Backen
4	4	MA-003	Alex	Meyer	IT-Support	1	2023-05-01	(NULL)	(NULL)	30,0	40	d	Fotografieren,Ga
5	5	MA-005	Thomas	Bauer	Marketing	4	2023-11-20	(NULL)	(NULL)	24,5	20	m	Sport & Fitness,M

In Anknüpfung an die letzte Unterrichtseinheit noch einmal ein Blick darauf, wer wann und was eingetragen hat.

13 interessen	14 angelegt_am	15 angelegt_von	16 letzte_aenderung_am	17 letzte_aenderung_von	18 ist_geloescht
Gaming,Musik	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
Lesen,Reisen	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
Kochen & Bakken	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
Fotografieren,Gaming	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
Sport & Fitness,Musik	2026-03-02 11:55:51	Admin_System	(NULL)	-	0

Check-Constraint

Jetzt müssen wir noch Sorge dafür tragen, dass bei dem wichtigen Bereich der Attribute gehalt, stundenlohn und wochenstunden keine Fehler durch Doppeleingabe passieren können.

Alles, was wir bisher getan haben, diente der technischen Absicherung unserer Datenbank. Nun wenden wir uns dem Bereich der so genannten „Business Rules“ zu. Hierbei geht es darum geschäftliche, nicht technische Regeln in der Datenbank abzubilden. Dies wird über ein so genanntes „Check-Constraint“ realisiert und direkt in der Tabelle mitarbeiter integriert. Hierzu ändern wir die Struktur der Tabelle mitarbeiter mittels des ALTER-Statements ab.

Das ist ein sehr wichtiger Schritt, um die Datenqualität langfristig zu sichern. Da wir nun exakt wissen, dass entweder ein Gehalt oder ein Stundenlohn (inkl. Stunden) vorhanden sein muss, können wir die Tabelle mit einem CHECK-Constraint "absichern".

```
1 ALTER TABLE mitarbeiter
2 ADD CONSTRAINT chk_vergütung_exklusiv
3 CHECK (
4     -- Fall 1: Festgehalt (Lohn und Stunden müssen leer sein)
5     (gehalt IS NOT NULL AND stundenlohn IS NULL AND wochenstunden IS NULL)
6     OR
7     -- Fall 2: Stundenlohn (Gehalt muss leer sein, Lohn und Stunden müssen gefüllt sein)
8     (gehalt IS NULL AND stundenlohn IS NOT NULL AND wochenstunden IS NOT NULL)
9 );
```


Das ALTER-Statement ist ohne Fehler durchgelaufen und hat die entsprechenden Einträge in der Datenbank und der Tabelle ergänzt.

Zum einen sehen wir den Eintrag unter dem Tab Prüfbedingungen (Abb.91),



Abb.91

zum anderen sehen wir die Veränderung im CREATE-Statement für die Tabelle mitarbeiter.

```
25  CONSTRAINT `chk_vergütung_exklusiv` CHECK (`gehalt` is not null and `stundenlohn` is null and `wochenstunden` is null or `gehalt` is null and `stundenlohn` is not null and `wochenstunden` is not null)
```

Warum dieser Constraint (auferlegter Zwang) so wichtig ist:

1. Keine "Doppel-Zahlungen": Er verhindert, dass ein Mitarbeiter versehentlich sowohl ein festes Monatsgehalt als auch einen Stundenlohn eingetragen bekommt.
2. Keine "Gratis-Arbeit": Er stellt sicher, dass ein Mitarbeiter nicht ohne jegliche Vergütungsangabe im System landet.
3. Vollständigkeit beim Lohn: Er erzwingt, dass beim Stundenlohn auch immer die Wochenstunden mit angegeben werden, da sonst keine Lohnabrechnung möglich wäre.

Wie es sich in der Praxis verhält:

Wenn du nun versuchst, ein INSERT oder UPDATE auszuführen, das gegen diese Regel verstößt, wird die Datenbank die Transaktion mit einer Fehlermeldung (z.B. Check constraint 'chk_vergütung_exklusiv' is violated) abbrechen.

Negativ- vs. Positivtest

Um sicherzustellen, dass unser neuer Constraint `chk_vergütung_exklusiv` wie eine unüberwindbare Mauer funktioniert, habe ich drei Test-Szenarien vorbereitet: zwei, die bewusst fehlschlagen müssen (Negativtests), und eines, das erfolgreich sein sollte (Positivtest).

1. Test: Versuch, beides einzutragen (Fehlschlag)

Hier versuchen wir, einem Mitarbeiter sowohl ein Gehalt als auch einen Stundenlohn zuzuweisen. Die Datenbank muss dies blockieren.

```
1  -- Dieser Befehl sollte einen Fehler auslösen (Constraint Violation)
2  INSERT INTO mitarbeiter (
3      mitarbeiter_nummer, vorname, nachname, abteilung,
4      eintritts_datum, gehalt, stundenlohn, wochenstunden,
5      geschlecht, angelegt_am, angelegt_von, letzte_aenderung_von
6  ) VALUES (
7      'TEST-001', 'Fehler', 'Beides', 'Testabteilung',
8      '2026-01-01', 5000.00, 25.00, 40, 'd',
9      NOW(), 'Test_User', '-'
10 );
```

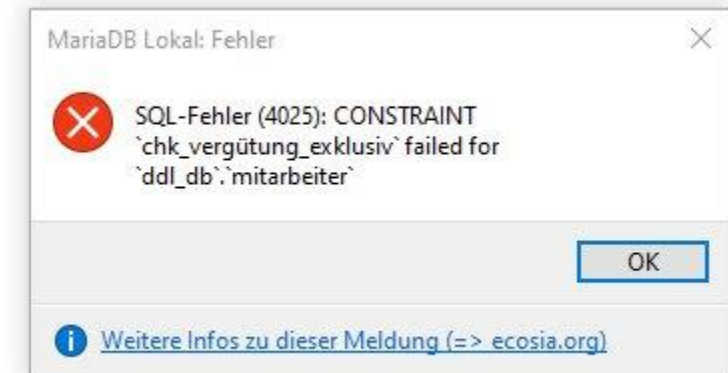


Das hat schon mal (nicht) geklappt.

2. Test: Versuch, gar nichts einzutragen

Hier versuchen wir, einen Mitarbeiter ohne jegliche Vergütung anzulegen. Auch das darf laut Constraint nicht passieren.

```
1  -- Dieser Befehl sollte ebenfalls einen Fehler auslösen
2  INSERT INTO mitarbeiter (
3      mitarbeiter_nummer, vorname, nachname, abteilung,
4      eintritts_datum, gehalt, stundenlohn, wochenstunden,
5      geschlecht, angelegt_am, angelegt_von, letzte_aenderung_von
6  ) VALUES (
7      'TEST-002', 'Fehler', 'Nichts', 'Testabteilung',
8      '2026-01-01', NULL, NULL, NULL, 'd',
9      NOW(), 'Test_User', '-'
10 );
```



Wie erwartet, MariaDB wirft eine Fehlermeldung.

3. Test: Korrekte Eingabe

Dieser Datensatz hält sich an die Regel (nur Gehalt, kein Lohn/Stunden) und wird akzeptiert.

```
1  -- Dieser Befehl wird erfolgreich ausgeführt
2  INSERT INTO mitarbeiter (
3      mitarbeiter_nummer, vorname, nachname, abteilung,
4      eintritts_datum, gehalt, stundenlohn, wochenstunden,
5      geschlecht, angelegt_am, angelegt_von, letzte_aenderung_von
6  ) VALUES (
7      'TEST-003', 'Erfolg', 'Gehalt', 'Testabteilung',
8      '2026-01-01', 4500.00, NULL, NULL, 'm',
9      NOW(), 'Test_User', '-'
10 );
```

6	6	TEST-003	Erfolg	Gehalt	Testabteilung	(NULL)	2026-01-01	(NULL)	4.500,0	(NULL)	(NULL)	m
---	---	----------	--------	--------	---------------	--------	------------	--------	---------	--------	--------	---

Bei den INSERT-Statements sieht es gut aus, dann sollten wir noch die UPDATE-Statements testen.

Attribute mit UPDATE manipulieren

Die Folge von UPDATE(); Die Änderung von Attribute-Werten. Beachte, dass unser Trigger im Hintergrund automatisch letzte_aenderung_am auf das aktuelle Datum setzt und die angelegt_...-Felder schützt.

Die Tabelle mitarbeiter vor dem UPDATE() ausgewählter Attribute.

	1 id	2 nachname	3 abteilung	4 wochenstunden	5 austritts_datum	6 angelegt_am	7 angelegt_von	8 letzte_aenderung_am	9 letzte_aenderung_von	10 ist_geloescht
1	1	Mustermann	IT-Leitung	(NULL)	(NULL)	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
2	2	Schmidt	Personalwesen	(NULL)	(NULL)	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
3	3	König	Marketing	(NULL)	(NULL)	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
4	4	Meyer	IT-Support	40	(NULL)	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
5	5	Bauer	Marketing	20	(NULL)	2026-03-02 11:55:51	Admin_System	(NULL)	-	0
6	6	Gehalt	Testabteilung	(NULL)	(NULL)	2026-03-02 13:16:32	Test_User	(NULL)	-	0


```
1  -- 1. Abteilungswechsel und Namensänderung (z.B. nach Heirat)
2  UPDATE mitarbeiter
3  SET nachname = 'Mustermann-Schulz',
4      abteilung = 'IT-Strategie',
5      letzte_aenderung_von = 'HR_Admin'
6  WHERE mitarbeiter_nummer = 'MA-001';
7
8  -- 2. Anpassung der Wochenstunden (für einen Lohnempfänger)
9  UPDATE mitarbeiter
10 SET wochenstunden = 35,
11     letzte_aenderung_von = 'Abteilungsleiter'
12 WHERE mitarbeiter_nummer = 'MA-005';
13
14 -- 3. Hinzufügen eines neuen Hobbys (Nutzung der SET-Funktionalität)
15 UPDATE mitarbeiter
16 SET interessen = 'Gaming,Musik,Reisen',
17     letzte_aenderung_von = 'Self_Service'
18 WHERE mitarbeiter_nummer = 'MA-003';
19
20 -- 4. Eintragung eines Austrittsdatums
21 UPDATE mitarbeiter
22 SET austritts_datum = '2026-12-31',
23     letzte_aenderung_von = 'HR_Admin'
24 WHERE mitarbeiter_nummer = 'MA-002';
25
26 -- 5. Logisches Löschen eines Mitarbeiters (Soft-Delete)
27 UPDATE mitarbeiter
28 SET ist_geloescht = TRUE,
29     letzte_aenderung_von = 'System_Audit'
30 WHERE mitarbeiter_nummer = 'MA-004';
```

Attribute nach zulässigen UPDATE()-Statements

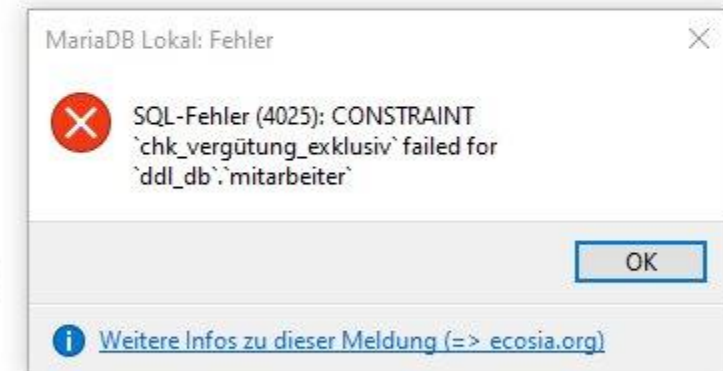
Unsere Tabelle mitarbeiter nach den Update()-Statements.

	1 id	2 nachname	3 abteilung	4 wochenstunden	5 austritts_datum	6 angelegt_am	7 angelegt_von	8 letzte_aenderung_am	9 letzte_aenderung_von	10 ist_geloescht
1	1	Mustermann-Schulz	IT-Strategie	(NULL)	(NULL)	2026-03-02 11:55:51	Admin_System	2026-03-02 13:39:27	HR_Admin	0
2	2	Schmidt	Personalwesen	(NULL)	2026-12-31	2026-03-02 11:55:51	Admin_System	2026-03-02 13:39:27	HR_Admin	0
3	3	König	Marketing	(NULL)	(NULL)	2026-03-02 11:55:51	Admin_System	2026-03-02 13:39:27	System_Audit	1
4	4	Meyer	IT-Support	40	(NULL)	2026-03-02 11:55:51	Admin_System	2026-03-02 13:39:27	Self_Service	0
5	5	Bauer	Marketing	35	(NULL)	2026-03-02 11:55:51	Admin_System	2026-03-02 13:39:27	Abteilungsleiter	0
6	6	Gehalt	Testabteilung	(NULL)	(NULL)	2026-03-02 13:16:32	Test_User	(NULL)	-	0

Unzulässige UPDATE()-Statements

Diese 3 Statements-Beispiele werden von der Datenbank abgelehnt, da sie gegen den CHECK-Constraint `chk_vergütung_exklusiv` verstoßen.

```
1  -- FEHLER 1: Versuch, einem Gehaltsempfänger zusätzlich einen Stundenlohn zu geben
2  -- (Verstoß: gehalt IS NOT NULL AND stundenlohn IS NOT NULL)
3  UPDATE mitarbeiter
4  SET stundenlohn = 45.00,
5      wochenstunden = 40,
6      letzte_aenderung_von = 'Admin_Fehler'
7  WHERE mitarbeiter_nummer = 'MA-001';
8
9  -- FEHLER 2: Versuch, bei einem Lohnempfänger das Gehalt zu füllen, ohne Lohn zu löschen
10 -- (Verstoß gegen die Exklusivität)
11 UPDATE mitarbeiter
12 SET gehalt = 5000.00,
13     letzte_aenderung_von = 'Admin_Fehler'
14 WHERE mitarbeiter_nummer = 'MA-003';
15
16 -- FEHLER 3: Versuch, alle Vergütungsfelder auf NULL zu setzen
17 -- (Verstoß: Weder Fall 1 noch Fall 2 des Constraints sind erfüllt)
18 UPDATE mitarbeiter
19 SET gehalt = NULL,
20     stundenlohn = NULL,
21     wochenstunden = NULL,
22     letzte_aenderung_von = 'Admin_Fehler'
23 WHERE mitarbeiter_nummer = 'MA-005';
```



DELETE() – Die Problematik des physikalischen Löschens

In professionellen Datenbankumgebungen ist ein einfaches DELETE FROM table WHERE id = X oft die risikoreichste Operation. Das hat primär drei Gründe:

1. Verletzung der Referenziellen Integrität

- Deine Tabelle mitarbeiter nutzt Fremdschlüssel-Beziehungen (FOREIGN KEY).
- Das Problem: Wenn du den Mitarbeiter mit der id = 1 (Max Mustermann-Schulz) löschst, hängen die Datensätze von Erika Schmidt und Klaus Meyer "in der Luft", da deren vorgesetzter_id auf eine ID verweist, die nicht mehr existiert.
- Die Folge: Das DBMS würde das Löschen verhindern (wegen ON DELETE RESTRICT), sofern die Constraints korrekt gesetzt sind. Wäre jedoch ON DELETE CASCADE aktiv, würden mit einem Befehl ganze Hierarchiebäume unwiederbringlich gelöscht werden.

Fremdschlüssel (1)						
+	Schlüsselname	Spalten	Referenztable	Fremdspalten	Beim UPDATE	Beim DELETE
+	fk_vorgesetzter	vorgesetzter_id	ddl_db.mitarbeiter	id	RESTRICT	RESTRICT

2. Verlust der Historisierung und Nachvollziehbarkeit

Datenbanken sind oft die "Single Source of Truth". Löscht man einen Datensatz physikalisch:

- Sind statistische Auswertungen für die Vergangenheit (z.B. Lohnkosten für das Vorjahr) nicht mehr korrekt möglich.
- Gehen Informationen für Audits verloren (Wer hat wann was gemacht?).
- In unserer Struktur sehen wir bereits Spalten wie `letzte_aenderung_am` – diese Metadaten wären bei einem DELETE ebenfalls sofort weg.

3. Gesetzliche Anforderungen (GoBD & DSGVO)

- GoBD*: Im geschäftlichen Umfeld müssen Buchungsbelege und personalkritische Daten oft 10 Jahre revisionssicher aufbewahrt werden. Ein DELETE widerspricht der Forderung nach Unveränderbarkeit bzw. Nachvollziehbarkeit.
- DSGVO: Hier gibt es zwar das "Recht auf Vergessenwerden", dies wird aber meist durch Anonymisierung oder Sperrung gelöst, nicht durch blindes Löschen, solange gesetzliche Aufbewahrungsfristen dagegenstehen.

*GoBD steht für „Grundsätze zur ordnungsmäßigen Führung und Aufbewahrung von Büchern, Aufzeichnungen und Unterlagen in elektronischer Form sowie zum Datenzugriff“. Es handelt sich um vom Bundesfinanzministerium herausgegebene Richtlinien für die digitale Buchführung in Deutschland, die steuerrelevante Belege revisionssicher, nachvollziehbar und zeitgerecht archivieren müssen.

Lösung mit Soft Delete

Um den Gefahren – die mit der Problematik des physikalischen Löschens verbunden sind – zu umgehen, setzt man in der Praxis fast immer auf das Konzept des Soft Deletes. Unsere Tabelle ist darauf bereits perfekt vorbereitet!

Das Konzept des Soft Deletes

Anstatt die Zeile aus dem Speicher zu entfernen, wird sie lediglich als "gelöscht" markiert.

Unsere Spalte: ist_geloescht (TINYINT/Boolean).

Der Vorgang: Ein Löschbefehl wird zu einem UPDATE.

```
1 UPDATE mitarbeiter m
2 SET m.ist_geloescht = 1
3 WHERE m.id = 6;
```

```
1 SELECT m.id, m.vorname, m.nachname, m.ist_geloescht
2 FROM mitarbeiter m;
```

mitarbeiter (6r × 4c)				
#	1 id	2 vorname	3 nachname	4 ist_geloescht
1	1	Max	Mustermann-Schulz	0
2	2	Erika	Schmidt	0
3	3	Sarah	König	1
4	4	Alex	Meyer	0
5	5	Thomas	Bauer	0
6	6	Erfolg	Gehalt	1

So sehen wir alle als gelöscht markierten Datensätze welche nach wie vor noch vorhanden sind.

Mit dem angepassten SELECT Statement sehen wir nur noch die Datensätze welche aktiv, das heißt nicht als gelöscht markiert sind.

Vorteil: Die referenzielle Integrität bleibt gewahrt. Verknüpfte Daten bleiben valide. Der Datensatz kann bei Bedarf (z.B. versehentliches Löschen) mit einem einfachen Update wiederhergestellt werden.

```
1 SELECT m.id, m.vorname, m.nachname, m.ist_geloescht
2 FROM mitarbeiter m
3 WHERE m.ist_geloescht = 1;
```

mitarbeiter (2r x 4c)				
#	1 id	2 vorname	3 nachname	4 ist_geloescht
1	3	Sarah	König	1
2	6	Erfolg	Gehalt	1

```
1 SELECT m.id, m.vorname, m.nachname, m.ist_geloescht
2 FROM mitarbeiter m
3 WHERE m.ist_geloescht = 0;
```

mitarbeiter (4r x 4c)				
#	1 id	2 vorname	3 nachname	4 ist_geloescht
1	1	Max	Mustermann-Schulz	0
2	2	Erika	Schmidt	0
3	4	Alex	Meyer	0
4	5	Thomas	Bauer	0

Lösung mit Hard Delete – START TRANSACTION

Das physikalische Löschen wird nur noch in Ausnahmefällen angewendet:

- Bereinigung von temporären Tabellen.
- Endgültiges Löschen nach Ablauf von gesetzlichen Haltefristen (Data Purging).
- Korrekte Umsetzung der DSGVO nach Ablauf aller Fristen.

Da es sich bei dem DELETE-Statement um einen destruktiven Prozess handelt ist es sicher eine gute Idee diesen als Transaktion auszuführen.

Kontrolle mit dem SELET-Statement.

```
1 START TRANSACTION;  
2  
3 DELETE FROM mitarbeiter m  
4 WHERE m.id = 2;
```

```
1 SELECT m.id, m.nachname  
2 FROM mitarbeiter m  
3 ORDER BY m.id;
```

mitarbeiter (5r × 2c)		
	1 id	2 nachname
1	1	Mustermann-Schulz
2	3	König
3	4	Meyer
4	5	Bauer
5	6	Gehalt

START TRANSACTION und ROLLBACK

Analogie: Gerade die Enter-Taste gedrückt und schon ruft der Chef an und teilt mir mit, dass der Mitarbeiter/Datensatz auf keinen Fall gelöscht werden darf. Kein Problem, wir haben ja in einer Transaktion gelöscht und noch nicht COMMITet. Also machen wir einen Rollback und schauen uns das Ergebnis an.

Glück gehabt, dass wir mit der erforderlichen Sorgfalt gearbeitet haben und die gelöschten Daten wieder herstellen konnten. Hätten wir das nicht getan wären die Daten unwiederbringlich verloren gewesen.

```
1  START TRANSACTION;
2
3  DELETE FROM mitarbeiter m
4  WHERE m.id = 2;

6  ROLLBACK;
```

```
1  SELECT m.id, m.nachname
2  FROM mitarbeiter m
3  ORDER BY m.id;
```

mitarbeiter (6r x 2c)		
	1 id	2 nachname
1	1	Mustermann-Schulz
2	2	Schmidt
3	3	König
4	4	Meyer
5	5	Bauer
6	6	Gehalt

"Bevor Daten physikalisch gelöscht werden, muss die referenzielle Integrität geprüft werden. Um Datenverlust zu vermeiden und die Revisionssicherheit (GoBD) zu garantieren, ist ein Soft-Delete-Verfahren (logisches Löschen über ein Flag) dem Hard-Delete (physisches Entfernen) vorzuziehen. Datenbank-Constraints schützen dabei vor inkonsistenten Zuständen."



Das Thema Referentielle Integrität ist ein Kernbestandteil der relationalen Datenbanktheorie:

Definition:

Die Referentielle Integrität ist eine Form der Datenintegrität, die sicherstellt, dass Beziehungen zwischen Tabellen konsistent bleiben. Sie besagt, dass

- ein Fremdschlüsselwert (Foreign Key) immer auf einen existierenden Primärschlüsselwert (Primary Key) der referenzierten Tabelle verweisen muss – oder er muss NULL sein (sofern erlaubt).

In einfachen Worten: Es darf keine "Waisen" geben. Ein Datensatz darf nicht auf etwas verweisen, das es gar nicht gibt.

Um die referentielle Integrität in einem DBMS (wie MariaDB, PostgreSQL oder Oracle) zu gewährleisten, werden Constraints (Einschränkungen) definiert. Diese greifen bei drei kritischen Operationen:

1. Beim Einfügen (INSERT)

Es ist nicht möglich, in die Tabelle mitarbeiter einen Datensatz einzufügen, dessen vorgesetzter_id eine Nummer enthält, die nicht als id in der Tabelle existiert.

- Beispiel: Du kannst keinen Mitarbeiter anlegen, der Untergeordnete von ID 99 ist, wenn es nur 6 Mitarbeiter gibt.

2. Beim Aktualisieren (UPDATE)

Ändert sich ein Primärschlüssel (was man in der Praxis vermeiden sollte), müssen die darauf verweisenden Fremdschlüssel nachgezogen werden.

- CASCADE: Die Änderung wird automatisch an alle Fremdschlüssel übertragen.
- RESTRICT / NO ACTION: Die Änderung des Primärschlüssels wird verboten, solange noch Fremdschlüssel darauf verweisen.

3. Beim Löschen (DELETE) – Besonders wichtig!

Dies ist der kritischste Punkt für die Datenkonsistenz. Wenn ein Datensatz gelöscht werden soll, auf den andere Tabellen (oder Zeilen) verweisen, bietet SQL verschiedene Strategien:

- **RESTRICT (Standard):** Das Löschen wird verhindert. (In deinem CREATE-Statement definiert: ON DELETE RESTRICT). Du kannst Chef Max Mustermann nicht löschen, solange Erika Schmidt ihm noch zugeordnet ist.
- **CASCADE: "Kaskadierendes Löschen".** Löschst du den Chef, werden automatisch auch alle seine Untergebenen gelöscht. (In der Personalverwaltung meist fatal!).
- **SET NULL:** Der Verweis beim Untergebenen wird auf NULL gesetzt. Der Mitarbeiter bleibt bestehen, hat aber keinen Vorgesetzten mehr.

RESTRICT in der Praxis

Unser Fremdschlüssel-Constraint steht auf RESTRICT und verhindert so das Löschen.

Glück gehabt, dass wir mit der erforderlichen Sorgfalt gearbeitet haben und die gelöschten Daten wieder herstellen konnten. Hätten wir das nicht getan wären die Daten unwiederbringlich verloren gewesen.

Schlüsselname	Spalten	Referenztabelle	Fremdspalten	Beim UPDATE	Beim DELETE
 fk_vorgesetzter	vorgesetzter_id	ddl_db.mitarbeiter	id	RESTRICT	RESTRICT

Wie auf nebenstehender Abbildung zu erkennen ist, hat der Mitarbeiter mit der ID 4 den Mitarbeiter mit der ID 5 als Untergebenen.

1

SELECT m.id, m.nachname, m.vorgesetzter_id

2

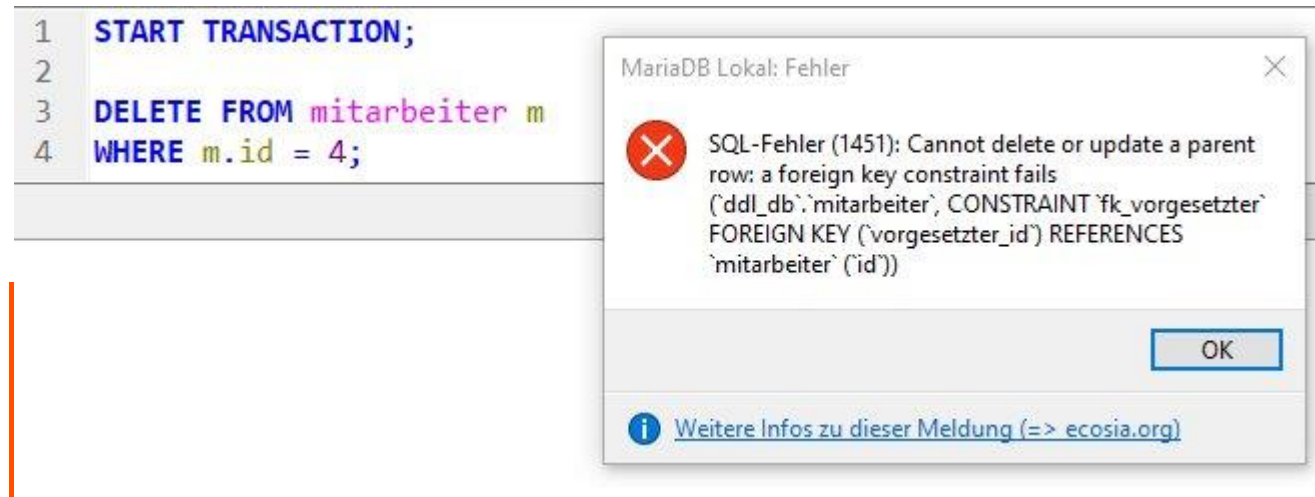
FROM mitarbeiter m

3

ORDER BY m.id;

mitarbeiter (6r x 3c)		
1 id	2 nachname	3 vorgesetzter_id
1	1 Mustermann-Schulz	(NULL)
2	2 Schmidt	1
3	3 König	(NULL)
4	4 Meyer	1
5	5 Bauer	4
6	6 Gehalt	(NULL)

Wenn wir jetzt den Mitarbeiter mit der ID 4 löschen wollen verletzen wir die Datenintegrität weil dann der Mitarbeiter mit der ID 5 keinen Vorgesetzten mehr hat und der Fremdschlüssel (FK) ohne zugehörigen Primärschlüssel (PK) in der Luft hängen würde.



Zu unserem Glück bewahrt uns das Fremdschlüssel-Constraint vor solchen katastrophalen Handlungen und stellt die Integrität der Daten in diesem Bereich sicher.

Die Frage nach der Sinnhaftigkeit: „Warum ist das wichtig?“

Es wird oft gefragt, welche Vorteile die Durchsetzung der referentiellen Integrität direkt im DBMS gegenüber einer Prüfung in der Anwendungssoftware (z.B. in Java oder PHP) hat:

- Zentralisierung:** Die Logik liegt direkt bei den Daten. Egal welche Applikation auf die Datenbank zugreift, die Regeln gelten immer.
- Fehlervermeidung:** Programmierfehler in der Applikation können die Datenbank nicht korrumpieren (keine "toten" Links).
- Performance:** Datenbanken sind darauf optimiert, diese Prüfungen hocheffizient beim Schreibvorgang durchzuführen.

Definition:

Eine Transaktion ist eine logische Einheit von Datenbankoperationen (INSERT, UPDATE, DELETE), die untrennbar zusammengehören. Die Transaktionsverarbeitung stellt sicher, dass diese Operationen entweder vollständig oder gar nicht ausgeführt werden.

Das ACID-Prinzip:

Um die Zuverlässigkeit zu garantieren, muss jede Transaktion vier Eigenschaften erfüllen:

- | | |
|-----------------------------------|--|
| A – Atomarität (Atomicity): | „Alles-oder-Nichts-Prinzip“. Scheitert ein einzelner Befehl innerhalb der Kette, wird die gesamte Transaktion rückgängig gemacht (Rollback). |
| C – Konsistenz (Consistency): | Vor und nach der Transaktion muss die Datenbank in einem gültigen Zustand sein (Einhaltung von Regeln wie unseren CHECK-Constraints). |
| I – Isolation (Isolation): | Gleichzeitige Transaktionen dürfen sich nicht gegenseitig beeinflussen. Ein Nutzer sieht keine „Zwischenstände“ eines anderen Nutzers. |
| D – Dauerhaftigkeit (Durability): | Sobald eine Transaktion mit Commit abgeschlossen ist, bleiben die Daten dauerhaft gespeichert, auch bei einem Systemausfall. |

Ohne Transaktionsverarbeitung und deren Schutzzweck drohen gravierende Datenfehler!

Der Schutzzweck:

verhindert Inkonsistenz.

Bei einer Umbuchung wird Geld an Stelle A abgezogen, aber ein Systemfehler verhindert das Gutschreiben an Stelle B. Die Transaktion sorgt dafür, dass das Geld bei A wieder erscheint (Rollback).

vermeidet Datenverlust.

Schutz bei Stromausfällen oder Programmabstürzen während eines Schreibvorgangs.

Mehrbenutzerbetrieb (Concurrency).

Verhindert, dass zwei Mitarbeiter gleichzeitig denselben Datensatz ändern und sich gegenseitig überschreiben (z. B. bei einer Gehaltsanpassung).

Der Autocommit-Status in MariaDB ist standardmäßig auf ON (1) gesetzt und steuert so, dass SQL-Anweisungen sofort permanent in die Datenbank geschrieben werden und nicht manuell bestätigt (COMMIT) werden müssen.

Folgend die unterschiedlichen Möglichkeiten, um den Autocommit-Status zu ändern:

1. Für die aktuelle Session (temporär)

Dies ist die gebräuchlichste Methode, um Autocommit nur für die aktuelle Verbindung zu deaktivieren.

```
SET autocommit = 0;           -- deaktiviert AUTOCOMMIT
SET autocommit = 1;           -- aktiviert AUTOCOMMIT
```

2. Für eine einzelne Transaktion (ohne SET)

Anstatt den Modus dauerhaft für die Session zu ändern, kann man eine Transaktion explizit starten. Dies deaktiviert Autocommit nur für diesen einen Vorgang. Dies ist die bevorzugte Methode, um sicherzustellen, dass man nicht vergisst, den Autocommit-Modus wieder einzuschalten.

```
START TRANSACTION;
-- Ihre SQL-Anweisungen hier
COMMIT; -- Zum permanenten Speichern
-- oder
ROLLBACK; -- Zum Verwerfen der Änderungen
```

1tes Test-Szenario: INSERT mit ROLLBACK

Wir versuchen, einen neuen Mitarbeiter anzulegen, brechen den Vorgang aber ab.

```
1  START TRANSACTION;
2
3  INSERT INTO mitarbeiter (
4      mitarbeiter_nummer, vorname, nachname, abteilung,
5      eintritts_datum, gehalt, geschlecht,
6      angelegt_am, angelegt_von, letzte_aenderung_von
7  ) VALUES (
8      'TRANS-001', 'Ghost', 'User', 'Test',
9      '2026-03-01', 5000.00, 'd',
10     NOW(), 'Admin', '-'
11 );
```

```
13 -- Jetzt prüfen wir: In dieser Session ist der User sichtbar
14 SELECT * FROM mitarbeiter WHERE mitarbeiter_nummer = 'TRANS-001';
15
```

mitarbeiter (1r × 18c)		mitarbeiter (0r × 18c)					
#	1 id	2 mitarbeiter_nummer	3 vorname	4 nachname	5 abteilung	6 vorgesetzter_id	7 eintritts_datum
1	7	TRANS-001	Ghost	User	Test	(NULL)	2026-03-01

```
16 -- Wir entscheiden uns um: Alles rückgängig machen!
17 ROLLBACK;
18
19 -- Finale Prüfung: Der User darf NICHT mehr existieren
20 SELECT * FROM mitarbeiter WHERE mitarbeiter_nummer = 'TRANS-001';
```

mitarbeiter (1r × 18c)		mitarbeiter (0r × 18c)	
1	id	2	mitarbeiter_nummer
3	vorname	4	nachname
5	abteilung	6	vorgesetzter_id
7	eintritts_datum		

2tes Test-Szenario: UPDATE mit Fehler (Die "Kettenreaktion")

Das ist der wichtigste Test. Wir versuchen zwei Updates. Das erste ist korrekt, das zweite provoziert einen Fehler (Constraint-Verletzung). In einer echten Transaktion muss das erste Update ebenfalls rückgängig gemacht werden.

Es gibt noch die Möglichkeit innerhalb von Transaktionen SAVEPOINTS (Sicherungspunkte) zu setzen. Damit kann man dann innerhalb einer Transaktion schrittweise zurück gehen (ROLLBACK TO SAVEPOINT name;). Dieses Verfahren empfiehlt sich für komplexe INSERT-, UPDATE- oder DELETE-Statements.

```
1 START TRANSACTION;
2
3 -- 1. Gültiges Update: Gehaltserhöhung für Max
4 UPDATE mitarbeiter SET gehalt = 7000.00, letzte_aenderung_von = 'Boss'
5 WHERE mitarbeiter_nummer = 'MA-001';
6
7 -- 2. UNGÜLTIGES Update: Wir versuchen, Sarah (MA-004) Gehalt UND Lohn zu geben
8 -- Dies wird fehlschlagen (Constraint Violation)
9 UPDATE mitarbeiter SET stundenlohn = 50.00, wochenstunden = 40
10 WHERE mitarbeiter_nummer = 'MA-004';
```

```
1 -- Transaktion starten
2 START TRANSACTION;
3
4 -- 1. Gültiges Update: Gehaltserhöhung für Max
5 UPDATE mitarbeiter
6 SET gehalt = 7000.00,
7     letzte_aenderung_von = 'Boss'
8 WHERE mitarbeiter_nummer = 'MA-001';
9
10 -- Savepoint setzen
11 SAVEPOINT nach_gehaltserhoehung;
12
13 -- 2. Ungültiges Update: Wir versuchen Sarah (MA-004) Gehalt (UND Lohn zu geben)
14 -- Dies wird fehlschlagen (Constraint Violation)
15 UPDATE mitarbeiter
16 SET stundenlohn = 50.00,
17     wochenstunden = 40
18 WHERE mitarbeiter_nummer = 'MA-004';
19
20 -- Teilweises ROLLBACK zum SAVEPOINT
21 ROLLBACK TO SAVEPOINT nach_gehaltserhoehung;
22
23 -- 3. Jetzt gültiges Update: Gehaltserhöhung für Sarah (MA-004)
24 UPDATE mitarbeiter
25 SET gehalt = 3500.00
26 WHERE mitarbeiter_nummer = 'MA-004';
27
28 -- Savepoint löschen (optional, gibt Ressourcen frei)
29 RELEASE SAVEPOINT nach_gehaltserhoehung;
30
31 -- Gesamte Transaktion dauerhaft speichern
32 COMMIT;
```

2tes Test-Szenario: Vor dem UPDATE

```
1 SELECT m.mitarbeiter_nummer,
2       m.nachname,
3       m.gehalt,
4       m.stundenlohn,
5       m.wochenstunden,
6       m.letzte_aenderung_am,
7       m.letzte_aenderung_von
8 FROM mitarbeiter m
9 WHERE m.mitarbeiter_nummer IN ('MA-001', 'MA-004');
```

mitarbeiter (2r x 7c)

	1 mitarbeiter_nummer	2 nachname	3 gehalt	4 stundenlohn	5 wochenstunden	6 letzte_aenderung_am	7 letzte_aenderung_von
1	MA-001	Mustermann-Schulz	6.500,0	(NULL)	(NULL)	2026-03-02 13:39:27	HR_Admin
2	MA-004	König	4.500,0	(NULL)	(NULL)	2026-03-02 13:39:27	System_Audit

2tes Test-Szenario: Das UPDATE()-Statement

```
1 START TRANSACTION;
2
3 -- 1. Gültiges Update: Gehaltserhöhung für Max
4 UPDATE mitarbeiter SET gehalt = 7000.00, letzte_aenderung_von = 'Boss'
5 WHERE mitarbeiter_nummer = 'MA-001';
6
7 -- 2. UNGÜLTIGES Update: Wir versuchen, Sarah (MA-004) Gehalt UND Lohn zu geben
8 -- Dies wird fehlschlagen (Constraint Violation)
9 UPDATE mitarbeiter SET stundenlohn = 50.00, wochenstunden = 40
10 WHERE mitarbeiter_nummer = 'MA-004';
11
```

MariaDB Lokal: Fehler

SQL-Fehler (4025): CONSTRAINT
'chk_vergütung_exklusiv' failed for
'ddl_db`.`mitarbeiter'

OK

Weitere Infos zu dieser Meldung (=> [ecosia.org](https://www.ecosia.org))

2tes Test-Szenario: Nach dem UPDATE

```
1 SELECT m.mitarbeiter_nummer,  
2        m.nachname,  
3        m.gehalt,  
4        m.stundenlohn,  
5        m.wochenstunden,  
6        m.letzte_aenderung_am,  
7        m.letzte_aenderung_von  
8 FROM mitarbeiter m  
9 WHERE m.mitarbeiter_nummer IN ('MA-001', 'MA-004');
```

mitarbeiter (2r x 7c)							
	1 mitarbeiter_nummer	2 nachname	3 gehalt	4 stundenlohn	5 wochenstunden	6 letzte_aenderung_am	7 letzte_aenderung_von
1	MA-001	Mustermann-Schulz	7.000,0	(NULL)	(NULL)	2026-03-02 14:38:06	Boss
2	MA-004	König	4.500,0	(NULL)	(NULL)	2026-03-02 13:39:27	System_Audit

2tes Test-Szenario: mit ROLLBACK

```
12 -- Da der zweite Befehl einen Fehler geworfen hat, machen wir den Rollback  
13 -- (In einer Applikation würde dies der Error-Handler tun)  
14 ROLLBACK;
```

2tes Test-Szenario: Daten nach dem ROLLBACK

```
1 SELECT m.mitarbeiter_nummer,
2      m.nachname,
3      m.gehalt,
4      m.stundenlohn,
5      m.wochenstunden,
6      m.letzte_aenderung_am,
7      m.letzte_aenderung_von
8 FROM mitarbeiter m
9 WHERE m.mitarbeiter_nummer IN ('MA-001', 'MA-004');
```

mitarbeiter (2r x 7c)							
	1 mitarbeiter_nummer	2 nachname	3 gehalt	4 stundenlohn	5 wochenstunden	6 letzte_aenderung_am	7 letzte_aenderung_von
1	MA-001	Mustermann-Schulz	6.500,0	(NULL)	(NULL)	2026-03-02 13:39:27	HR_Admin
2	MA-004	König	4.500,0	(NULL)	(NULL)	2026-03-02 13:39:27	System_Audit

Was kann zusammengefasst gesagt werden?

Folgend einige Antworten auch die berechtigte Frage eines jeden Entwicklers: „Worauf muss ich bei der Formulierung einer Transaktionen achten?“

Atomarität	Wenn ein Teil der Transaktion scheitert (wie im UPDATE-Beispiel), darf der Rest nicht in der Datenbank bleiben.
Isolation	Während die Transaktion läuft (START TRANSACTION), sehen andere Benutzer deine Änderungen noch nicht (Dirty Reads werden verhindert).
Bestätigung	Erst mit dem Befehl COMMIT; werden die Daten dauerhaft auf die Festplatte geschrieben.

Was ist ein Index?

Ein Index ist eine zusätzliche Datenstruktur (meist ein B-Baum), die die Suche nach Datensätzen in einer Datenbank beschleunigt. Man kann ihn sich wie das Stichwortverzeichnis am Ende eines Fachbuchs vorstellen: Statt das ganze Buch Seite für Seite durchzulesen (Full Table Scan), schlägt man den Begriff nach und springt direkt zur richtigen Seitenzahl.

Allgemein Optionen Indizes (4) Fremdschlüssel (1) Prüfbedingungen (1) Partitionen

Neu

Entfernen

Leeren

Auf

Ab

Name	Typ / Länge
PRIMARY KEY	PRIMARY
mitarbeiter_nummer	UNIQUE
fk_vorgesetzter	KEY
nachname	KEY

Spalten:

Neu

Entfernen

Auf

Ab

#	Name	Datentyp	Länge/SET	Vorzeichenlos
1	id	INT	11	☐
2	mitarbeiter_nummer	VARCHAR	20	☐
3	vorname	VARCHAR	50	☐
4	nachname	VARCHAR	50	☐
5	abteilung	VARCHAR	50	☐
6	vorgesetzter_id	INT	11	☐

Index-Typ	Beschreibung	Verwendung / Ziel
Primary Key	Eindeutiger Identifikator pro Zeile. Er darf niemals NULL sein. Pro Tabelle gibt es nur einen.	Identifikation (z.B. die id in unserer mitarbeiter-Tabelle).
Unique Index	Garantiert, dass alle Werte in einer Spalte unterschiedlich sind (außer Null).	Integrität (z.B. für die mitarbeiter_nummer, damit keine Nummer doppelt vergeben wird).
Normaler Key (index)	Ein einfacher Suchoptimierer ohne Einschränkung für die Datenwerte.	Geschwindigkeit (z.B. auf dem Feld nachname, um Mitarbeiter schneller zu finden).
Fulltext Index	Spezialindex für große Textblöcke. Er ermöglicht die Suche nach Wörtern innerhalb langer Texte.	Volltextsuche (z.B. um in der Spalte interessen effizienter nach Begriffen zu suchen).

Wofür werden Indizes benötigt?

Performance (Geschwindigkeit)	Ohne Index muss die Datenbank bei jeder Abfrage jede einzelne Zeile prüfen. Bei Millionen von Datensätzen dauert das Sekunden statt Millisekunden.
Eindeutigkeit (Datenintegrität)	Primary und Unique Indizes verhindern logische Fehler, wie doppelte Personalnummern oder doppelte E-Mail-Adressen.
Sortierung	Indizes helfen der Datenbank, Daten schneller sortiert (ORDER BY) oder gruppiert (GROUP BY) auszugeben.
Verknüpfung (Joins)	Fremdschlüssel-Beziehungen (wie unsere vorgesetzter_id) arbeiten intern mit Indizes, um Tabellen blitzschnell miteinander zu verbinden.

Der Zielkonflikt

Indizes machen die Datenbank beim Lesen schneller, aber beim Schreiben langsamer. Grund: Bei jedem INSERT, UPDATE oder DELETE muss die Datenbank nicht nur die Tabelle ändern, sondern auch den Index-Baum aktualisieren.

Deshalb gilt: So viele Indizes wie nötig, so wenige wie möglich!

Es ist wichtig, nicht nur zu wissen, dass Indizes die Suche beschleunigen, sondern auch, welcher Algorithmus für welches Szenario am effizientesten ist. Die Wahl des richtigen Index-Typs beeinflusst die Performance bei Millionen von Datensätzen massiv.

Folgend die Übersicht der drei gängigsten Algorithmen:

Funktionsweise	Stärken	Anwendung
1) B-Tree (Balanced Tree)		
Die Daten werden in einer baumartigen, balancierten Struktur sortiert gespeichert. Die Suche beginnt an der Wurzel und hangelt sich über Zweige bis zu den Blättern (den eigentlichen Daten) vor.	Bereichsabfragen: Er ist exzellent für Operatoren wie <code>></code>, <code><</code>, <code>BETWEEN</code> oder <code>LIKE 'ABC%'</code>. Sortierung: Da die Daten im Baum bereits sortiert vorliegen, unterstützt er <code>ORDER BY</code> extrem effizient.	Ideal für unsere Spalten <code>nachname</code> , <code>gehalt</code> oder <code>eintritts_datum</code> .

Funktionsweise	Stärken	Anwendung
----------------	---------	-----------

2) Hash-Index

Ein Hash-Index arbeitet nach dem Prinzip einer Schlüssel-Wert-Tabelle (Key-Value-Pair).

Der Spaltenwert wird durch eine Hash-Funktion in eine eindeutige Zahl (Hash-Wert) umgerechnet, die direkt auf die Speicheradresse zeigt.

Stärken:

Punktabfragen:

Er ist unschlagbar schnell bei exakten Übereinstimmungen (WHERE id = 5).

Konstante Zeit:

Die Suche dauert theoretisch immer gleich lang, egal wie groß die Tabelle ist.

Schwächen:

Völlig unbrauchbar für Bereichsabfragen (>) oder Sortierungen, da der Hash-Wert nichts über die Reihenfolge der Daten aussagt.

Oft genutzt für eindeutige Identifikatoren wie die mitarbeiter_nummer, wenn nur nach exakten Werten gesucht wird.

Funktionsweise	Stärken	Anwendung
----------------	---------	-----------

3) R-Tree (Rectangle Tree / Spatial Index)

Der R-Tree wird für geografische oder räumliche Daten benötigt.

Er organisiert Daten in sich überschneidenden Rechtecken (Minimum Bounding Rectangles). Statt nach einem Wert sucht er nach einer Koordinate innerhalb eines Bereichs.	Umkreissuche: „Finde alle Filialen im Umkreis von 10km“. Geometrie: Überlappungsprüfungen von Flächen.	In unserer mitarbeiter-Tabelle aktuell nicht verbaut, aber essenziell für GPS-Koordinaten oder Kartenanwendungen.
--	---	---

Algorithmus	Suchart	Merkmal
B-Tree	Gleichheit & Bereich	• Allrounder • unterstützt Sortierung.
Hash	Nur Gleichheit	• Schnellster bei exakten Treffern • keine Sortierung
R-Tree	Räumlich / Geografisch	• optimiert für 2D/ 3D-Koordinaten

Der B-Tree Index in der Realität

Um den Effekt eines B-Tree Index auf das Feld gehalt zu verdeutlichen, schauen wir uns an, wie die Datenbank ohne und mit Index arbeitet.

Das dazugehörige Szenario:

Wir suchen alle Mitarbeiter, die ein Gehalt zwischen 4000 € und 5000 € beziehen.

Hierzu verwenden wir das Schlüsselwort EXPLAIN vor dem SELECT-Statement.

```
1 SELECT m.mitarbeiter_nummer, m.nachname, m.gehalt
2 FROM mitarbeiter m
3 WHERE gehalt BETWEEN 4000 AND 5000;
```

mitarbeiter (3r × 3c) mitarbeiter (1r × 10c)			
#	1 mitarbeiter_nummer	2 nachname	3 gehalt
1	MA-002	Schmidt	4.800,0
2	MA-004	König	4.500,0
3	TEST-003	Gehalt	4.500,0

```
5 EXPLAIN SELECT m.mitarbeiter_nummer, m.nachname, m.gehalt
6 FROM mitarbeiter m
7 WHERE gehalt BETWEEN 4000 AND 5000;
```

mitarbeiter (3r × 3c) mitarbeiter (1r × 10c)										
	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SIMPLE	m	ALL	(NULL)	(NULL)	(NULL)	(NULL)	6	Using where

1. Ohne Index: Der "Full Table Scan"

Ohne Index muss die Datenbank jede einzelne Zeile der Tabelle von oben nach unten lesen.

Vorgehensweise:

Die Datenbank prüft Zeile 1, dann Zeile 2, dann Zeile 3... bis zum Ende.

Aufwand:

Bei 100.000 Mitarbeitern sind 100.000 Leseoperationen nötig.

Dauer:

Je nach Festplattengeschwindigkeit kann dies spürbare Millisekunden, Sekunden oder sogar Minuten dauern. Je nach Datenmenge.

```
5 EXPLAIN SELECT m.mitarbeiter_nummer, m.nachname, m.gehalt
6 FROM mitarbeiter m
7 WHERE gehalt BETWEEN 4000 AND 5000;
```

mitarbeiter (3r x 3g)										mitarbeiter (1r x 10g)									
	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra									
1	1	SIMPLE	m	ALL	(NULL)	(NULL)	(NULL)	(NULL)	6	Using where									

Die "gezielte Suche"

Zuerst legen wir den Index an.

Vorgehensweise:

Die Datenbank nutzt nun den sortierten B-Tree.

- Sie springt in den Baum und findet in wenigen Schritten (Logarithmische Zeit) den ersten Wert ≥ 4000 .
- Da der B-Tree sortiert ist, liest sie einfach die folgenden Einträge nacheinander aus, bis sie den Wert 5000 überschreitet.

Aufwand:

Statt 100.000 Zeilen zu prüfen, muss die Datenbank vielleicht nur 5–10 Knoten im Baum und die Trefferliste lesen.

Dauer:

Nahezu sofort (nahe 0 ms).

```
1 CREATE INDEX IF NOT EXISTS idx_mitarbeiter_gehalt ON mitarbeiter(gehalt);
```

```
5 EXPLAIN SELECT m.mitarbeiter_nummer, m.nachname, m.gehalt
6 FROM mitarbeiter m
7 WHERE gehalt BETWEEN 4000 AND 5000;
```

mitarbeiter (3r x 3q)		mitarbeiter (1r x 10q)							
1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1 SIMPLE	m	range	idx_mitarbeiter_gehalt	idx_mitarbeiter_gehalt	7	(NULL)	3	Using index condition

Abb.502

Der B-Tree Index in der Realität (Fortsetzung) – Der Vorher-Nachher-Vergleich mit EXPLAIN

In der Praxis nutzt man den Befehl EXPLAIN, um die Strategie der Datenbank (den "Execution Plan") sichtbar zu machen.

5

EXPLAIN SELECT m.mitarbeiter_nummer, m.nachname, m.gehalt

6

FROM mitarbeiter m

7

WHERE gehalt BETWEEN 4000 AND 5000;

mitarbeiter (3r × 3c)		mitarbeiter (1r × 10c)								
	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SIMPLE	m	ALL	(NULL)	(NULL)	(NULL)	(NULL)	6	Using where

Status	EXPLAIN Ausgabe (Spalte type)	Bedeutung
Vorher	ALL	Full Table Scan (schlecht bei großen Datenmengen)
Nachher	range	Index-Bereichssuche (sehr effizient für BETWEEN)

5

EXPLAIN SELECT m.mitarbeiter_nummer, m.nachname, m.gehalt

6

FROM mitarbeiter m

7

WHERE gehalt BETWEEN 4000 AND 5000;

mitarbeiter (3r × 3c)		mitarbeiter (1r × 10c)								
	1 id	2 select_type	3 table	4 type	5 possible_keys	6 key	7 key_len	8 ref	9 rows	10 Extra
1	1	SIMPLE	m	range	idx_mitarbeiter_gehalt	idx_mitarbeiter_gehalt	7	(NULL)	3	Using index condition

Was kann zusammengefasst gesagt werden?

Ein Index beschleunigt SELECT-Abfragen massiv, da die Komplexität von $O(n)$ (linear) auf $O(\log n)$ (logarithmisch) sinkt. Allerdings führt jeder Index zu einem Mehraufwand bei INSERT- und UPDATE-Operationen, da der Baum aktuell gehalten werden muss.

Die O-Notation (Big O Notation) beschreibt in der Informatik, wie sich der Aufwand (Zeit oder Speicherplatz) eines Algorithmus verhält, wenn die Menge der Eingabedaten (n) wächst. Kurz gesagt: Sie misst nicht die exakte Zeit in Sekunden, sondern die Effizienz oder das "Wachstumstempo".

Notation	Name	Bedeutung	Beispiel
$O(1)$	Konstant	Der Aufwand ist immer gleich groß, egal wie groß n ist.	Zugriff auf ein Element in einem Array per Index.
$O(\log n)$	Logarithmisch	Der Aufwand steigt nur sehr langsam an.	Suche in einem B-Tree Index.
$O(n)$	Linear	Der Aufwand wächst exakt proportional zur Datenmenge.	Ein Full Table Scan (jede Zeile einmal prüfen).
$O(n^2)$	Quadratisch	Der Aufwand vervierfacht sich, wenn sich die Daten verdoppeln,	Zwei verschachtelte Schleifen (z.B. einfacher Sortieralgorithmus).



Vielen Dank für eure Aufmerksamkeit.